

# SERVE — ANTIGRAVITY MASTER DEVELOPMENT PLAN

You are working on **SERVE**, a 2-developer food ordering and canteen management system.

Your job is to implement the project **incrementally, safely, and without creating Git conflicts**.

---

## 1. PROJECT ARCHITECTURE

The repository contains three independent applications:

```
Serve/  
├─ student-app/           # Developer 1 – Flutter + Dart  
├─ staff-dashboard/       # Developer 2 – React 19 + TypeScript +  
Vite  
└─ backend/              # Developer 2 – Node.js + TypeScript +  
Express + Socket.IO
```

Database:

```
PostgreSQL  
  ↓  
Prisma ORM  
  ↓  
Express Backend
```

External services:

```
Firebase Authentication  
Firebase Cloud Messaging  
Razorpay  
AWS
```

The backend is the **single source of truth**.

Frontend applications must NEVER directly modify database state.

---

## 2. ABSOLUTE AGENT RULES

These rules have higher priority than convenience.

### Rule 1 — Folder ownership

Developer 1 owns:

```
student-app/**
```

Developer 2 owns:

```
backend/**  
staff-dashboard/**
```

Shared files:

```
README.md  
.gitignore
```

docs/DEV\_PLAN.md (this document – both may read it; edits require agreement between both developers)

Do NOT modify another developer's owned folder.

Do NOT "helpfully" fix files outside your assigned ownership.

If a required change belongs to another developer's folder:

1. Do not modify it.
  2. Describe the required change.
  3. Continue using mocks/interfaces where possible.
- 

## 3. GIT STRATEGY

Branches:

main  
develop  
feature/\*

main = stable release branch.

develop = integration branch.

feature/\* = implementation branches.

NEVER commit directly to:

main  
develop

Every feature must use a feature branch.

Examples:

feature/student-auth  
feature/student-menu  
feature/student-cart  
feature/student-orders  
feature/student-payment

feature/backend-schema  
feature/backend-auth  
feature/backend-menu  
feature/backend-orders  
feature/backend-payment  
feature/backend-sockets

feature/staff-shell  
feature/staff-orders  
feature/staff-menu  
feature/staff-pickup

Before beginning work:

```
git checkout develop
git pull origin develop
git checkout -b feature/<feature-name>
```

Before opening a PR:

```
git fetch origin
git rebase origin/develop
```

Resolve conflicts ONLY inside your own feature branch.

Use:

```
git push --force-with-lease
```

when required after rebasing your own feature branch.

Use **Squash Merge** for:

```
feature/* → develop
```

Do not create unnecessary merge commits.

Only merge:

```
develop → main
```

when a complete milestone has been tested.

---

## 4. FIRST ACTION — REPOSITORY SAFETY

Before implementing features:

### Developer 1

Create the Flutter project:

```
flutter create student-app
```

Verify:

```
flutter doctor  
flutter run
```

Set up:

- Flutter structure
- routing/navigation
- theme
- reusable widgets
- API service layer
- model layer
- Firebase client configuration

### Developer 2

First inspect and stabilize the backend.

IMPORTANT:

The repository currently contains:

```
prisma@8.0.0-rc.15  
@prisma/client@7.10.0
```

These versions **MUST** be aligned before serious schema work.

Do not proceed with mismatched Prisma packages.

Then:

- configure local PostgreSQL
- configure Prisma
- run migrations

- create backend folder architecture
  - configure Firebase Admin
  - configure environment variables
  - prepare API routes
  - prepare Socket.IO
  - replace placeholder routes gradually
- 

## 5. BACKEND STRUCTURE

Use:

```
backend/src/  
├─ controllers/  
├─ routes/  
├─ services/  
├─ middleware/  
├─ validators/  
├─ sockets/  
├─ utils/  
└─ server.ts
```

Responsibilities:

### **routes/**

HTTP endpoint definitions.

### **controllers/**

Request/response handling.

### **services/**

Business logic.

### **middleware/**

Authentication, authorization, error handling.

### **validators/**

Request validation.

### **sockets/**

Realtime event handling.

NEVER put large business logic directly inside route files.

---

## 6. DATABASE OWNERSHIP

ONLY Developer 2 modifies:

```
backend/prisma/schema.prisma
```

Required core models:

User

FoodItem  
Order  
OrderItem  
Payment  
Notification

Required functionality should support:

User roles:  
STUDENT  
STAFF

Order lifecycle:

PENDING  
CONFIRMED  
PREPARING  
READY  
COMPLETED  
CANCELLED

Payment state and order state MUST remain separate.

Additional schema support may include:

CanteenStatus  
Announcement  
estimatedReadyAt  
pickupWindowStart  
pickupWindowEnd  
pickupPin  
capacity configuration

Do not introduce unnecessary database tables.

Prefer the simplest schema that correctly supports the product.

After schema changes:

```
npx prisma generate  
npx prisma migrate dev --name <description>
```

Commit migration files.

Never silently change database behavior without documenting the API impact.

---

## 7. API CONTRACT FIRST

Before frontend/backend integration, agree on exact request and response structures.

This is the shared interface between developers. **Developer 1 builds against these exact mock shapes. Developer 2 must preserve these shapes once implementation becomes real** — a shape change is a conversation and a version bump on this document, not a silent break.

Never invent random JSON structures in frontend or backend code — use exactly what's below.

**GET /api/canteen/status**

**Auth:** Student or Staff token

```
// 200
{ "status": "OPEN", "message": "Orders open until 9 PM",
  "updatedAt": "2026-09-16T18:00:00Z" }
```

#### GET /api/menu

**Auth:** Student or Staff token

```
// 200
{
  "items": [
    { "id": "uuid", "name": "Veg Puff", "price": 30, "isAvailable":
true, "prepTime": 5, "imageUrl": "https://..." }
  ]
}
```

#### GET /api/menu/:id

**Auth:** Student or Staff token

```
// 200
{ "id": "uuid", "name": "Veg Puff", "description": "...", "price":
30, "isAvailable": true, "prepTime": 5, "imageUrl": "https://..." }
// 404
{ "error": "ITEM_NOT_FOUND" }
```

#### POST /api/orders

**Auth:** Student token

```
// Request
{ "items": [ { "foodItemId": "uuid", "quantity": 2 } ] }

// 201
{ "orderId": "uuid", "orderNumber": "ORD-1234", "totalAmount": 90,
"status": "PENDING" }

// 400
{ "error": "ITEM_UNAVAILABLE", "itemId": "uuid" }

// 409
{ "error": "CANTEEN_CLOSED" }
```

Server always recalculates totalAmount from current DB prices — never trusts a client-submitted amount.

#### GET /api/orders/my-orders

**Auth:** Student token

```
// 200
{
  "orders": [
    { "orderId": "uuid", "orderNumber": "ORD-1234", "status":
"COMPLETED", "totalAmount": 90, "createdAt": "2026-09-16T18:00:00Z"
  }
  ]
}
```

#### GET /api/orders/:id

**Auth:** Student token (own order only) or Staff token

```
// 200
```

```

    {
      "orderId": "uuid", "orderNumber": "ORD-1234", "status":
"PREPARING",
      "estimatedReadyAt": "2026-09-16T18:20:00Z",
      "pickupWindowStart": "2026-09-16T18:15:00Z", "pickupWindowEnd":
"2026-09-16T18:30:00Z",
      "pickupPin": "4821",
      "items": [ { "name": "Veg Puff", "quantity": 2, "priceAtTime": 30
} ]
    }

// 404
{ "error": "ORDER_NOT_FOUND" }

```

## POST /api/payments/create-order

**Auth:** Student token

```

// Request
{ "orderId": "uuid" }

// 200
{ "razorpayOrderId": "order_xyz", "amount": 9000, "currency": "INR",
"keyId": "rzp_test_xxx" }

```

amount is in paise. keyId is the public key — safe to expose.  
 RAZORPAY\_KEY\_SECRET never appears in any response.

## POST /api/payments/verify

**Auth:** Student token

```

// Request
{ "orderId": "uuid", "razorpayOrderId": "order_xyz",
"razorpayPaymentId": "pay_xyz", "razorpaySignature": "sig..." }

// 200
{ "status": "CONFIRMED" }

// 400
{ "error": "SIGNATURE_INVALID" }

// 409 (idempotency – safe to call twice)
{ "error": "ALREADY_VERIFIED" }

```

## POST /api/payments/webhook

**Auth:** Razorpay webhook signature (not a user token)

```

// Request: raw Razorpay webhook payload
// 200
{ "received": true }

```

Must be idempotent — check `Payment.status` before writing, since this  
 can fire for a payment already confirmed via `/verify`.

## PATCH /api/staff/orders/:id/status

**Auth:** Staff token only

```

// Request (allowed values: PREPARING, READY)
{ "status": "PREPARING" }

// 200

```

```
{ "orderId": "uuid", "status": "PREPARING" }

// 403
{ "error": "FORBIDDEN" }

// 400
{ "error": "INVALID_TRANSITION" }
```

#### **POST /api/staff/orders/:id/verify-pickup**

**Auth:** Staff token only

```
// Request
{ "pin": "4821" }
// or: { "qrPayload": "..." }

// 200
{ "orderId": "uuid", "status": "COMPLETED" }

// 400
{ "error": "PIN_MISMATCH" }

// 409 (already collected – prevents double-completion)
{ "error": "ALREADY_COMPLETED" }
```

#### **PATCH /api/staff/menu/:id/availability**

**Auth:** Staff token only

```
// Request
{ "isAvailable": false }

// 200
{ "id": "uuid", "isAvailable": false }
```

#### **PATCH /api/staff/canteen/status**

**Auth:** Staff token only

```
// Request
{ "status": "PAUSED", "message": "Kitchen at capacity, resuming in
10 min" }

// 200
{ "status": "PAUSED" }
```

---

## **8. DEVELOPMENT PHASES**

### **PHASE 0 — FOUNDATION**

#### **Developer 1**

Build:

- Flutter app
- routing
- theme
- reusable components
- authentication screens
- API service abstraction

- models

## Developer 2

Build:

- Express structure
- PostgreSQL
- Prisma
- Firebase Admin
- auth middleware
- role middleware
- API route skeletons (returning the exact mock JSON from Section 7)
- Socket.IO structure

Goal:

Both projects should run independently.

---

# PHASE 1 — AUTH + MENU

## Developer 1

Implement:

Login  
Home  
Menu  
Food Details  
Availability UI

Use mock API responses matching Section 7 when backend endpoints are not ready.

## Developer 2

Implement:

Firebase token verification  
STUDENT / STAFF authorization  
User model  
FoodItem model  
GET /api/menu  
GET /api/menu/:id

Build the initial staff dashboard shell simultaneously.

Goal:

Student → Login → Menu  
Staff → Login → Dashboard

---

# PHASE 2 — CART + ORDER CREATION

## Developer 1

Implement:

Cart  
Quantity updates  
Price calculation UI  
Checkout  
Order confirmation  
Order number display

## Developer 2

Implement:

Order  
OrderItem  
Order creation API  
Server-side price validation  
Availability validation  
Order number generation

CRITICAL:

The server is responsible for calculating the final order amount.

Never trust prices sent by Flutter.

Goal:

Student  
→ Menu  
→ Cart  
→ Place Order  
→ Backend  
→ Order Created  
→ Order Number Returned

---

# PHASE 3 — STAFF ORDER OPERATIONS

## Developer 1

Build:

Order Tracking  
Order History

## Developer 2

Build:

Incoming Orders  
Order Details  
Start Preparing  
Mark Ready  
Mark Completed  
Order Search

State transitions must be validated server-side.

Example:

PENDING → CONFIRMED  
CONFIRMED → PREPARING  
PREPARING → READY

READY → COMPLETED

Invalid transitions must return an error.

---

## PHASE 4 — RAZORPAY

Razorpay secrets **MUST** exist only on the backend.

Flow:

```
graph TD
    Student --> CreateOrder[Create Order]
    CreateOrder --> Backend[Backend creates Razorpay order]
    Backend --> FlutterCheckout[Flutter opens Razorpay Checkout]
    FlutterCheckout --> RazorpayResult[Razorpay returns payment result]
    RazorpayResult --> FlutterBackend[Flutter sends result to backend]
    FlutterBackend --> BackendVerify[Backend verifies signature]
    BackendVerify --> PaymentSuccess[Payment = SUCCESS]
    PaymentSuccess --> OrderConfirmed[Order = CONFIRMED]
```

Implement webhook handling as a backstop.

Payment verification must be idempotent.

Do not mark an order confirmed based only on client-side success.

Never expose:

RAZORPAY\_KEY\_SECRET

to Flutter or React.

Only the public Razorpay key may be exposed to the client.

---

## 9. REALTIME SYSTEM

Use Socket.IO.

The backend emits events.

Clients receive events.

Clients do NOT directly emit domain-state changes.

**Socket authentication:** the client must pass its Firebase ID token when opening the connection (e.g. as the auth payload in the Socket.IO handshake). The backend verifies this token — the same way the REST auth middleware does — *before* allowing the socket to join any room. An unauthenticated or invalid-token connection must be rejected at connect time, not allowed to connect and then blocked per-event. This prevents any client from joining another student's student:{userId} room or the staff room without a valid staff-role token.

Rooms:

staff  
student:{userId}

Events:

canteen:status\_changed  
order:created  
order:status\_changed  
order:ready  
menu:item\_updated  
announcement:created

Example:

Staff marks order READY  
↓  
Backend updates database  
↓  
Backend emits order:status\_changed  
↓  
Student UI updates  
↓  
Backend triggers FCM notification  
  
Backend remains the source of truth.

---

## 10. STAFF DASHBOARD

Developer 2 implements:

Login  
Dashboard  
New Orders  
Preparing Orders  
Ready Orders  
Completed Orders  
Order Details  
Start Preparing  
Mark Ready  
Pickup Verification  
Menu Management  
Availability Toggle  
Food Creation  
Food Editing  
Order Search  
Basic Analytics

Keep analytics lower priority than the main order loop.

The complete order workflow must work before spending significant time on analytics.

---

## 11. STUDENT APP

Developer 1 implements:

Login  
Home  
Canteen Status  
Menu  
Food Details  
Cart

Checkout  
Razorpay  
Order Confirmation  
Order Tracking  
Order History  
Notifications  
Profile

Important UI states:

Loading  
Success  
Empty  
Error  
Offline/network failure  
Unavailable item  
Payment failure  
Order failure

Never implement only the happy path.

---

## 12. PICKUP FLOW

Student sees:

Order Number  
Pickup information  
Ready status

Staff verifies the order.

Verification may use:

Order Number  
PIN  
QR

Backend validates the pickup.

Only the backend can transition:

READY → COMPLETED

Wrong verification must fail safely.

Repeated pickup attempts must not complete an already completed order again.

---

## 13. NOTIFICATIONS

Firebase Cloud Messaging.

Trigger notifications for:

Order Ready  
Canteen Status Changes

The frontend should gracefully handle:

notification received while app open  
notification received in background  
notification tap

---

## 14. CONCURRENCY + DATA SAFETY

The application must work when multiple students order simultaneously.

Backend must prevent:

- Unavailable food being purchased
- Incorrect totals
- Duplicate orders
- Duplicate payment confirmation
- Invalid status transitions
- Duplicate pickup completion

Use server-side validation and database constraints wherever appropriate.

---

## 15. ANTIGRAVITY CODING RULES

Every time you implement a feature:

### Step 1

Inspect the existing code before creating new files.

### Step 2

Reuse existing architecture where possible.

Do NOT rewrite working architecture merely because another framework or folder structure looks cleaner.

### Step 3

Implement only the requested feature.

Do not perform unrelated refactors.

### Step 4

Run the application.

For Flutter:

```
flutter analyze
flutter test
flutter run
```

For React/backend:

```
npm run dev
npm run build
```

and run the relevant tests/linting if configured.

### Step 5

Fix errors before committing.

## Step 6

Review the Git diff.

Check that ONLY owned files changed.

## Step 7

Commit using Conventional Commits.

Examples:

```
feat(student-auth): add Firebase login flow
feat(student-menu): add menu availability state
feat(backend-orders): add order creation endpoint
feat(staff-orders): add preparing and ready actions
fix(backend-payment): prevent duplicate payment verification
```

---

# 16. NEVER DO THESE THINGS

NEVER:

- commit directly to main
  - commit directly to develop
  - modify another developer's folder
  - modify Prisma schema as Developer 1
  - modify Flutter code as Developer 2
  - hardcode secrets
  - expose Razorpay secret keys
  - trust client-side prices
  - trust client-side order status
  - bypass Firebase authorization
  - create random API response shapes
  - add unnecessary dependencies
  - rewrite the project architecture without a requirement
  - perform unrelated refactors
  - delete working code without understanding it
  - create huge commits containing unrelated work
  - merge code that has not been locally tested
  - allow a Socket.IO connection to join a room before its token is verified
- 

# 17. INTEGRATION CHECKPOINTS

Integration happens after each major phase.

## Checkpoint 1

Authentication works

Menu works

## Checkpoint 2

Cart

→ Order

→ Order Number

## Checkpoint 3

Order  
→ Staff Dashboard  
→ Preparing  
→ Ready

## Checkpoint 4

Razorpay  
→ Backend verification  
→ Confirmed order

## Checkpoint 5

Socket.IO  
→ Live student updates  
→ Staff updates  
→ FCM

## Checkpoint 6

Complete:

Login  
→ Browse  
→ Cart  
→ Pay  
→ Order  
→ Staff receives  
→ Preparing  
→ Ready  
→ Notification  
→ Pickup verification  
→ Completed

---

# 18. DEFINITION OF DONE

A feature is NOT complete until:

- implementation works
  - correct folder ownership is maintained
  - API contract (Section 7) is respected exactly
  - authentication/authorization is correct
  - loading state exists
  - error state exists
  - empty state exists where applicable
  - local build succeeds
  - tests/checks pass
  - Git diff contains only relevant changes
  - commit follows Conventional Commits
  - feature branch is ready for PR
- 

# 19. PRIORITY ORDER

Build in this order:

P0  
Foundation  
Authentication  
Menu

Cart  
Orders  
Staff order management

P1  
Razorpay  
Realtime updates  
Order tracking  
Pickup verification  
FCM

P2  
Analytics  
Advanced capacity monitoring  
Nice-to-have UX improvements

Do NOT build P2 features while the core order-to-pickup flow is broken.

---

## 20. FINAL AGENT BEHAVIOR

You are not allowed to assume that another developer's work exists just because the architecture document mentions it.

Inspect the repository first.

Respect current code.

Build incrementally.

Keep changes isolated.

Keep interfaces stable.

Prefer small, testable changes over large rewrites.

When another developer's work is required, create or use the agreed API contract (Section 7) rather than editing their folder.

The project's most important objective is:

A reliable end-to-end food ordering flow  
with clean separation between the  
Student App, Staff Dashboard, and Backend.

Before every commit, verify:

OWNED FILES ONLY  
↓  
BUILD PASSES  
↓  
TESTS/CHECKS PASS  
↓  
NO SECRETS  
↓  
API CONTRACT PRESERVED  
↓  
SMALL COMMIT  
↓  
FEATURE BRANCH